VINUNIVERSITY

Tutorial Lab 09: Introduction to graphs and basic graph algorithms.

May 11 2026

College of Engineering and Computer Science,
Object-Oriented Programming & Data Structures Spring

0. Introduction

A **Graph** is a non-linear data structure used to represent relationships between different objects. Graphs are commonly used in many real-world applications such as:

- Social media networks
- Google Maps and GPS systems
- Computer and communication networks
- Airline route systems
- Recommendation systems

A graph mainly consists of two components:

- **Vertices (Nodes)** – represent objects or locations
- **Edges** – represent connections between vertices

Example:

$$V = \{A, B, C, D\}$$

$$E = \{(A, B), (A, C), (B, D)\}$$

Here:

- V represents the set of vertices
- E represents the set of edges

The edge (A, B) means vertex A is connected to vertex B.

1. Vertex and Edge

A **Vertex** (also called a node) is a single point in a graph. An **Edge** is a connection between two vertices.

Vertices are used to store data, while edges represent relationships between them.

For example:

- In Facebook, users are vertices and friendships are edges
- In Google Maps, cities are vertices and roads are edges

Example

Vertices:

$$A, B, C$$

Edges:

$$(A, B), (B, C)$$

This means:

- A is connected to B
- B is connected to C

Java Implementation

```java
public class GraphBasics {
    public static void main(String[] args) {

        String[] vertices = {"A", "B", "C"};

        String[][] edges = {
            {"A", "B"},
            {"B", "C"}
        };

        System.out.println("Vertices:");
        for(String v : vertices) {
            System.out.println(v);
        }

        System.out.println("Edges:");
        for(String[] e : edges) {
            System.out.println(e[0] + " -> " + e[1]);
        }
    }
}
```

The program stores vertices and edges separately and prints them.

2. Directed and Undirected Graph

Graphs can be classified based on edge direction.

Undirected Graph

In an undirected graph, edges do not have direction. The connection works both ways.

If:

$$(A, B)$$

then:

$$A \leftrightarrow B$$

This means:

- A can reach B
- B can also reach A

Example applications:

- Friendship networks
- Two-way roads

Directed Graph

In a directed graph, edges have a specific direction.

If:

$$(A, B)$$

then:

$$A \rightarrow B$$

but not necessarily:

$$B \rightarrow A$$

Example applications:

- Instagram followers
- One-way roads
- Website links

Java Example

```java
public class DirectedUndirected {

    public static void main(String[] args) {

        // Undirected edge
        System.out.println("A <-> B");

        // Directed edge
        System.out.println("A -> B");
    }
}
```

This program simply demonstrates the difference between directed and undirected edges.

3. Graph Representation using Edge List

An **Edge List** stores all graph edges as pairs of vertices.

Each row represents one edge connection.

This representation is simple and easy to implement.

Example

$$(A, B), (A, C), (B, D)$$

This means:

- A is connected to B
- A is connected to C
- B is connected to D

Java Implementation

```java
public class EdgeListExample {

    public static void main(String[] args) {

        int[][] edges = {
            {0, 1},
            {0, 2},
            {1, 3}
        };

        System.out.println("Edge List:");

        for(int[] edge : edges) {
            System.out.println(edge[0] + " - " + edge[1]);
        }
    }
}
```

The program stores edges inside a 2D array and prints each edge connection.

4. Adjacency Matrix

An **Adjacency Matrix** represents a graph using a 2D array.

- Rows represent source vertices
- Columns represent destination vertices

If two vertices are connected:

$$1$$

Otherwise:

$$0$$

This method is easy for checking whether an edge exists between two vertices.

Example

$$\begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

Explanation:

- Vertex 0 is connected to 1 and 2
- Vertex 1 is connected to 0
- Vertex 2 is connected to 0

Java Implementation

```java
public class AdjacencyMatrix {
    public static void main(String[] args) {
        int[][] graph = {
            {0, 1, 1},
            {1, 0, 0},
            {1, 0, 0}
        };
        System.out.println("Adjacency Matrix:");

        for(int i = 0; i < graph.length; i++) {
            for(int j = 0; j < graph[i].length; j++) {
                System.out.print(graph[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

This program stores the graph using a matrix and prints all connections.

5. Adjacency List

An **Adjacency List** stores neighbors of each vertex using lists.

Each vertex keeps a list of connected vertices.

This representation uses less memory compared to adjacency matrices.

Example

$$A : B, C$$

$$B : A$$

$$C : A$$

Explanation:

- Vertex A is connected to B and C
- Vertex B is connected to A
- Vertex C is connected to A

Java Implementation

```java
import java.util.*;

public class AdjacencyList {

    public static void main(String[] args) {

        ArrayList<ArrayList<Integer>> graph =
                new ArrayList<>();

        for(int i = 0; i < 3; i++) {
            graph.add(new ArrayList<>());
        }

        graph.get(0).add(1);
        graph.get(0).add(2);

        graph.get(1).add(0);
        graph.get(2).add(0);

        for(int i = 0; i < graph.size(); i++) {
            System.out.println(i + " -> " + graph.get(i));
        }
    }
}
```

The program creates a graph using ArrayLists and prints neighboring vertices.

6. Degree of a Vertex

The **Degree** of a vertex is the number of edges connected to it.

For undirected graphs:

$$\text{Degree} = \text{number of connected edges}$$

A larger degree means the vertex has more connections.

Example

If:

$$A \rightarrow B$$

$$A \rightarrow C$$

Then degree of A is:

$$2$$

because two edges are connected to A.

Java Implementation

```java
public class VertexDegree {

    public static void main(String[] args) {

        int[][] graph = {
            {0,1,1},
            {1,0,0},
            {1,0,0}
        };

        int degree = 0;

        for(int j = 0; j < graph[0].length; j++) {
            degree += graph[0][j];
        }

        System.out.println("Degree of vertex 0: "
                           + degree);
    }
}
```

The program calculates the degree of vertex 0 by counting connected edges.